Contents

The signal model — the algebra of the chain **1**
1. The transmitter — `s(t)` . . . 1
2. Propagation — one source to one receiver . . . 2
   2.1 Emission time (the light-time solve) . . . 2
   2.2 Earth rotation (Sagnac) . . . 2
   2.3 Amplitude — free-space path loss + antenna . . . 2
   2.4 The received phase . . . 2
3. Superposition — the incident field . . . 3
4. The receiver front-end — operators on `r(t)` . . . 3
   4.1 Antenna (per source) — `G_ant` . . . 3
   4.2 The oscillator (per source, via `δ_o`) — `θ_{osc}(t)` . . . 4
   4.3 RF/IF gain + AGC (post-sum) — `g` . . . 4
   4.4 Nonlinearity (post-sum) — IP3 . . . 4
   4.5 Thermal noise (post-sum) — `n(t)` . . . 4
   4.6 The ADC — **the collapse** . . . 4
5. The complete forward model . . . 5
6. The inverse — what an estimator does . . . 5
7. Glossary . . . 5
8. Equation ↔ code map . . . 6

The signal model — the algebra of the chain

This is the mathematical specification the code implements. The guiding principle: **everything is a continuous function of time; the only collapse to discrete samples is the ADC.** The transmitter is a function `s(t)`; propagation, the oscillator, the antenna and the front-end are *operators* on it; sampling + quantisation is the single discrete, lossy step. Because every operator is exact and stateless-in-form, an *estimator* can run the same algebra **backwards** (§6).

The numbered equations **(E1)–(E10)** each cite the code that implements them; §8 collects the equation ↔ code map. Symbols are in the glossary (§7).

---

1. The transmitter — `s(t)`

A transmitter is a **stateless complex-baseband function** of emission time (`src/signal/mod.rs::SignalModel`) riding a nominal RF carrier `f_tx`:

$$s_{\mathrm{RF}}(t) \;=\; \Re\Big[\, s(t)\, e^{\,j2\pi f_{tx} t} \,\Big], \qquad s(t) \in \mathbb{C}.$$

`s(t)` is the *information*; `f_tx` is the carrier. Concrete models define `s(t)`:

signal	`s(t)`
CW tone (offset `f_b`)	$s(t) = e^{\,j2\pi f_b t}$
BPSK (bits `b_k`, rate `R`)	$s(t) = b_{\lfloor Rt \rfloor} \in \{\pm 1\}$
GPS C/A (PRN, NAV `d`)	$s(t) = c_{\mathrm{PRN}}(t)\, d(t) \in \{\pm 1\}$

The crucial property: **`s` is never pre-sampled.** Doppler, delay and clock all act by changing *where* `s` is evaluated, not by resampling a buffer — which is what keeps the model exact.

---

2. Propagation — one source to one receiver

2.1 Emission time (the light-time solve)

A sample the receiver records at its own time `t` was emitted earlier, at the **emission time** `τ(t)`, because the signal travelled the slant range at `c`. With the source at `P_s` and the receiver at `P_o`, `τ` solves the implicit equation (`channel::solve_emission_times`, fixed-point Newton):

$$\boxed{\tau(t) \;+\; \frac{\left\|\mathbf{P}_s(\tau) - \mathbf{P}_o(t)\right\|}{c} \;=\; t} \tag{E1}$$

code — `channel::solve_emission_times_for` (the per-sample Newton loop).

i.e. emission time + light-travel = reception time. Everything dynamic — Doppler, code-rate Doppler, sample-skew — falls out of `τ(t)` varying with `t`; there is no separate "apply Doppler" step.

2.2 Earth rotation (Sagnac)

`P_s(τ)` and `P_o(t)` are ECEF positions at *different* instants, but the geometric path must be measured in one frame. The source is rotated into the reception-frame by Earth's rotation over the transit, `Δ=t-τ` (linearised, exact to sub-mm):

$$\tilde{\mathbf{P}}_s = R_3(\omega_\oplus\,\Delta)\,\mathbf{P}_s(\tau) \;\approx\; \left(P_{s,x} + \omega_\oplus\Delta\,P_{s,y},\;\; P_{s,y} - \omega_\oplus\Delta\,P_{s,x},\;\; P_{s,z}\right). \tag{E2}$$

code — the `wtau/sx_rot/sy_rot` block inside `channel::solve_emission_times_for`.

The range used everywhere below is this Sagnac-corrected slant range, $r(t) = \|\tilde{\mathbf{P}}_s - \mathbf{P}_o\|$. Omitting it costs ~tens of ns — significant for a timing reference. (The OD side, `common.emission_ranges`, solves the *same* equation — reception- vs emission-time geometry was the light-time bug that cost ~50 m of along-track until fixed.)

2.3 Amplitude — free-space path loss + antenna

$$A(t) \;=\; \underbrace{\frac{\lambda}{4\pi\,r(t)}\sqrt{P_{tx}}}_{\text{FSPL (volts)}} \cdot \underbrace{\sqrt{G_{\text{ant}}(\theta_s(t))}}_{\text{rx antenna}}, \qquad \lambda = \frac{c}{f_{tx}}. \tag{E3}$$

code — `amp_factor/amplitude` (FSPL) × `geom_gain` (antenna √G), in `channel::accumulate_transmitter_co` mix loop.

`P_tx` is EIRP (W); the `λ/4πr` factor is the free-space field amplitude; √ because amplitude, not power. `G_ant` is the receiver antenna's power gain at the source's arrival elevation `θ_s(t)` (§4.1). lipika amplitudes are in **√W**, so `|·|²` is power directly.

2.4 The received phase

The phase of one source at receiver sample time `t` (LO `f_lo`):

$$\boxed{\varphi(t) \;=\; \underbrace{\arg s(\tau)}_{\text{baseband}} \;-\; \underbrace{2\pi f_{tx}\,(t-\tau)}_{\text{carrier delay}} \;+\; \underbrace{2\pi\,(f_{tx}-f_{lo})\,t}_{\text{IF / de-ramp}} \;-\; \underbrace{2\pi f_{lo}\,\delta_o(t)}_{\text{clock / LO}}} \tag{E4}$$

code — the `phase = neg_tau_src*(delay+delta_t) + tau_if*t_local` line in `channel::accumulate_transmitter_contribution` (`neg_tau_src=−2πf_tx`, `tau_if=2π(f_tx−f_lo)`, `delta_t=δ_o`).

Reading the terms:

- **baseband** `arg s(τ)` — the message, sampled at emission time.

- **carrier delay** `−2πf_tx(t−τ)` — the carrier phase accumulated over the light path. Its time-derivative *is* the carrier Doppler; it emerges automatically as `τ` walks with `t`. Note `t−τ = r/c + δ_o` (delay + clock).
- **IF / de-ramp** `2π(f_tx−f_lo)t` — when the receiver tunes its LO away from the source carrier, the residual sits at the intermediate frequency `f_tx−f_lo` in baseband. For an on-tune receiver (`f_lo=f_tx`) this term vanishes.
- **clock / LO** `−2πf_lo·δ_o(t)` — the receiver oscillator. **This single term is both the timing error and the LO phase noise**: the LO is derived from the node oscillator, so its phase error at frequency `f_lo` is exactly `2πf_lo·δ_o(t)` (see §4.2). This is why clock and LO are one object.

For an on-tune receiver this collapses to the form in the code:

$$\varphi(t) = \arg s(\tau) \;-\; 2\pi f_{tx}\Big(\tfrac{r(t)}{c} + \delta_o(t)\Big).$$

So one source contributes the continuous complex sample $A(t)\,|s(\tau)|\,e^{j\varphi(t)}$.

---

3. Superposition — the incident field

A receiver band sees the **sum over all in-band sources** (the `bands_overlap` gate decides "in-band"):

$$\boxed{\; r(t) \;=\; \sum_{i \in \text{sources}} A_i(t)\, s_i\big(\tau_i(t)\big)\, e^{j\varphi_i(t)} \;} \tag{E5}$$

> **code** — `pipeline::generate_band` sums each transmitter's `accumulate_transmitter_contribution` into the band buffer `acc` (fixed order ⇒ deterministic). `bands_overlap` gates the source set.

This is still an exact continuous function. Horizon occlusion enters here: a source below the receiver's local horizon is dropped (`A_i \to 0`).

---

4. The receiver front-end — operators on `r(t)`

The front-end is a composition of operators (`src/rx`). Some act **per source** (woven into §2, because they depend on each source's geometry); the rest act on the **summed band** `r(t)`. The chain, in order:

4.1 Antenna (per source) — `G_ant`

A gain on the arrival direction. The chosen self-phased QFH is azimuthally symmetric, so gain depends on elevation `θ` only:

$$G_{\text{ant}}(\theta)\;[\text{dBi}] = \begin{cases} G_{\text{horiz}} + (G_{\text{peak}} - G_{\text{horiz}})\,\theta/90^\circ & \theta \ge 0 \\ G_{\text{horiz}} - \text{F/B} & \theta < 0 \text{ (back-lobe)} \end{cases} \tag{E6}$$

applied as `√G` to each source's amplitude (§2.3). `θ` is the same arrival angle the horizon mask uses.

> **code** — `rx::Antenna::amplitude_at`; folded into the `geom_gain` pass of `channel::accumulate_transmitter_`

4.2 The oscillator (per source, via δ_o) — θ_{osc}(t)

The node has **one** oscillator phase process. Its fractional-frequency noise `y(t)` integrates to the **time error** `δ_o(t)` that appears in `φ` (§2.4):

$$\delta_o(t) \;=\; \underbrace{c_0}_{\text{offset}} + \underbrace{m}_{\text{drift}}\,t + \underbrace{\varepsilon_{\text{wFM}}(t)}_{\text{white FM (Allan }\sigma_1\text{)}} + \underbrace{\varepsilon_{\text{fFM}}(t)}_{\text{flicker FM (Allan floor)}} \tag{E7}$$

> **code** — `clock::NodeClockSpec` (the parameters), `clock::ClockStream` (the streaming `δ_o(t)`: deterministic part + white-FM Wiener walk), `clock::Flicker` (the flicker-FM floor).

- `c₀` — static bias (per-node, unobservable absolute; a nuisance in the OD).
- `m` — static fractional-frequency offset (ppm-scale drift).
- `ε_wFM` — white-FM **Wiener walk**, calibrated so $\sigma_y(\tau{=}1\text{s}) = \sigma_1$; Allan $\propto \tau^{-1/2}$.
- `ε_fFM` — **flicker FM** (1/f), a flat Allan **floor** — the oscillator-grade knob (TCXO ≈ 1e-9…1e-10, OCXO ≈ 1e-12…1e-13). Built as a sum of log-spaced AR(1) relaxations, self-calibrated (`clock::Flicker`).

The LO phase error is `2πf_lo·δ_o(t)` — the same `δ_o`. So **clock = LO**: one process drives both the timing (δ in the delay) and the carrier phase noise.

4.3 RF/IF gain + AGC (post-sum) — g

A linear scalar (dB) lifting the weak (FSPL-attenuated) signal into the ADC range: $y \mapsto g\,y$.

4.4 Nonlinearity (post-sum) — IP3

Memoryless third-order LNA/mixer model with input intercept power `P_{\text{IP3}}`:

$$y \;\mapsto\; y\Big(1 - \tfrac{4}{3}\,\frac{|y|^2}{P_{\text{IP3}}}\Big). \tag{E8}$$

> **code** — `rx::PostOp::Nonlinearity`.

Small-signal linear; gain-compresses and produces IM3 at strong input. The binding spec for terrestrial rejection. (`P_IP3` from the dBm datasheet value, direct since amplitude² is power.)

4.5 Thermal noise (post-sum) — n(t)

Additive complex AWGN at the receiver floor $P_n = k_B T B$ (`B` = sample rate): $y \mapsto y + n,\ n \sim \mathcal{CN}(0, P_n)$.

4.6 The ADC — the collapse

The only discrete, lossy, non-invertible step: sample on the nominal grid `t_n=n/f_s` (aperture jitter is a time-warp of this grid), clip to the full scale `±V_{fs}`, and quantise to `b` bits:

$$x[n] \;=\; Q_b\big(\text{clip}_{\pm V_{fs}}(y(t_n))\big), \qquad Q_b(v) = \Delta\,\text{round}(v/\Delta),\ \ \Delta = \frac{V_{fs}}{2^{b-1}-1}. \tag{E9}$$

> **code** — `rx::PostOp::Adc`.

---

5. The complete forward model

Putting it together — what lipika computes for output sample `n` of a band:

$$\boxed{x[n] \;=\; Q_b\!\left(\,\mathrm{clip}\!\left[\,\mathcal{N}\!\Big(g\sum_i A_i(t_n)\,s_i(\tau_i)\,e^{j\varphi_i(t_n)}\Big) \;+\; n[n]\,\right]\right)} \qquad t_n = \tfrac{n}{f_s} \tag{E10}$$

code — `pipeline::generate_band` per band: the `Σ` (E5) + `add_awgn_at_power` + `RxChain::apply_post_sum` (gain, NL, ADC), then `sink.push`.

Everything inside `Q_b(clip(·))` is an exact continuous expression evaluated at `t_n`; `Q_b` is the single collapse. With an unconfigured front-end (`g=1`, no NL, ideal ADC) this reduces to the plain `Σ` — which is why adding the front-end leaves old scenes byte-identical.

---

6. The inverse — what an estimator does

Because every operator above is exact and the lossy ones (NL, thermal, quantisation) are well-characterised noise, an estimator can run the **same algebra backwards** on `x[n]`, inverting the *invertible* operators and treating the rest as noise:

- **De-rotation** = inverse of §2.4's carrier+IF terms: multiply by `e^{-j[2π(f_tx-f_lo)t - 2πf_tx r̂/c]}` using a *predicted* range `r̂` (from ephemeris+position, or the orbit fit) → leaves the slow residual `-2πf·δ_o + noise`.
- **Code-phase prediction** = inverse of the §2.1 time-warp: predict `τ` from the broadcast ephemeris + surveyed position → the measured-minus-predicted code shift is the clock.
- **Clock recovery** = estimating `δ_o(t)` (E7): common-mode across all visible SVs (geometry wiped), code resolves integer-ns, carrier carries the ps fraction.

Because forward and inverse are the *same* `φ`, `r`, `δ`, a simulator and an estimator built on this algebra cannot drift apart — and a result proved against the forward geometry (e.g. the light-time term, E1–E2) transfers directly to the inverse.

---

7. Glossary

symbol	meaning
`t`	receiver-local reception time of a sample
`τ(t)`	emission time (light-time solve, §2.1)
`s(t)`	transmitter complex baseband (§1)
`f_tx`, `f_lo`	source RF carrier; receiver band LO
`f_s`, `B`	band sample rate (= noise bandwidth)
`P_s`, `P_o`	source / observer ECEF position
`r(t)`	Sagnac-corrected slant range
`c`, `λ`, `ω⊕`	speed of light; wavelength `c/f_tx`; Earth rotation rate
`P_tx`	transmit EIRP (W); lipika amplitude² is power
`G_ant(θ)`	rx antenna power gain at elevation `θ`
`δ_o(t)`	node clock/oscillator time error (§4.2)
`c₀`, `m`	clock static offset; fractional-frequency drift
`σ_1`	white-FM Allan deviation at `τ=1` s
`ε_wFM`, `ε_fFM`	white-FM walk; flicker-FM floor
`g`, `P_IP3`	RF/IF gain; input third-order intercept (W)
`P_n=k_BTB`	thermal noise power

symbol	meaning
V_{fs}, b, Δ	ADC full scale; bits; LSB
Q_b, n (index)	b-bit quantiser; output sample index

---

8. Equation ↔ code map

Each formula and the function that implements it. Source files are under `src/`; every function below carries a `(E…)` back-reference in its doc-comment.

eq	what	code
E1	emission-time (light-time) solve	`channel::solve_emission_times_for`
E2	Sagnac (Earth-rotation) correction	same loop (`wtau`/`sx_rot`/`sy_rot`)
E3	amplitude — FSPL × antenna √G	`channel::accumulate_transmitter_contributi` (`amp_factor`, `geom_gain`)
E4	received phase $\varphi(t)$	`channel::accumulate_transmitter_contributi` (the `phase = …` line)
E5	superposition $r(t)$	`pipeline::generate_band` (sum of contributions)
E6	antenna gain $G_{ant}(\theta)$	`rx::Antenna::amplitude_at`
E7	oscillator $\delta_o(t)$	`clock::{NodeClockSpec, ClockStream, Flicker}`
E8	IP3 nonlinearity	`rx::PostOp::Nonlinearity`
E9	ADC collapse (clip + quantise)	`rx::PostOp::Adc`
E10	full forward $x[n]$	`pipeline::generate_band` (Σ + noise + `RxChain::apply_post_sum`)

The reverse links live in the code: each listed function's doc-comment names its equation, so `grep -rn "SIGNAL_MODEL" src/` walks from code back to the maths.